

NexusAI

INTERVIEW PREPARATION HANDBOOK

Questions an interviewer will actually ask about this project — with answers grounded in YOUR codebase, including the hard questions about its real weaknesses.

COVERAGE

- | | |
|---------------------------------|-----------------------------------|
| 1. System Design & Architecture | 5. Database & State |
| 2. Real-Time & WebSockets | 6. AI & Blockchain |
| 3. Trading Engine & OMS | 7. Scaling & Production |
| 4. Security & Authentication | 8. Hard Questions About YOUR Code |
| 9. Behavioral / Project Story | |

How To Use This Handbook

Every answer here maps to code that actually exists in your repository, so you can defend each claim by opening the file. The most valuable section is Section 8 — the questions about your project's genuine weaknesses. Interviewers probe for self-awareness; being the one who already knows the flaws (and the fix) is what separates a strong candidate from an average one.

Answer framing rule

For each answer: (1) state what you did, (2) say why, (3) name the trade-off, (4) say what you would do at 100x scale. Interviewers reward the trade-off and the scale answer most.

SECTION 1 — SYSTEM DESIGN & ARCHITECTURE

The opener. They want to know you can describe the whole system before drilling in.

Q1. Walk me through the architecture of NexusAI end-to-end.

A React 19 + Vite SPA talks to a Node/Express API over REST and to a Socket.IO server over WebSockets. The backend has three long-lived singleton services: MarketDataService (the single source of truth for prices, polling Yahoo Finance), PaperTradingEngine (in-memory order execution with async flush to MongoDB), and BrokerGateway (an adapter layer that routes to Zerodha/Upstox/Angel One or a mock sandbox). MongoDB/Mongoose stores users, accounts, positions and orders. Razorpay handles wallet funding, Gemini powers the AI copilot, and a Solidity contract on Polygon logs predictions for tamper-proof verification. Timers in index.js drive market-data polling, heatmap deltas, and admin telemetry broadcasts.

Follow-up to expect: Where is the single point of failure, and how would you remove it?

Q2. Why did you centralize pricing in one service instead of fetching per-route?

Early on, different routes generated prices independently (some with Math.random), so the chart price, the execution price, and the portfolio valuation disagreed — a correctness bug in a finance app. I made MarketDataService a singleton that every consumer reads from, so there is exactly one authoritative price per symbol. It also lets me batch one Yahoo Finance query for all symbols (avoiding rate limits) and apply caching, staleness checks, and backoff in one place.

Follow-up to expect: What happens when you run two server instances?

Q3. What are the main trade-offs in your design?

Singletons give me simplicity and a clean single-source-of-truth, but they bind shared state to one process, so horizontal scaling needs Redis and external coordination. In-memory execution gives me very low latency but risks data loss on crash between flushes. Polling Yahoo Finance is simple and free but is not true real-time and can rate-limit — a production system would use a licensed streaming feed.

SECTION 2 — REAL-TIME & WEBSOCKETS

They will push on scale, fan-out, and what happens when clients misbehave.

Q4. How does the real-time price feed work?

MarketDataService polls Yahoo Finance every 5s during market hours (30s outside) and updates an in-memory cache. A separate 1s timer in index.js reads the cache and broadcasts batched updates via `io.emit('price_update', updates)`. Clients that want a specific stock join a room (`subscribe_stock`), so granular feeds like microstructure only go to subscribers. Batching many symbols into one packet cuts socket overhead.

Follow-up to expect: Why broadcast on a timer instead of pushing on every cache change?

Q5. How do you stop a malicious or buggy client from overwhelming the socket server?

Several guards in the connection handler: I throttle reconnection floods (if an IP reconnects more than 10 times within a second I disconnect it), I validate every subscribe payload (must be a string, max 30 chars) and de-duplicate room joins with a per-socket Set, I cap message size at 1MB, and a 5-minute interval cleans up stale sockets and old reconnection-tracking entries. Socket.IO ping/pong (25s interval, 60s timeout) detects dead connections.

Q6. Your timers run inside the app process. What breaks at scale?

With multiple instances, every instance runs its own polling and broadcast timers, so you get duplicate Yahoo Finance load and clients receive duplicated emits. The fix is a Redis Socket.IO adapter so a publish fans out across instances, plus electing a single leader (or a separate worker / cron) to own the polling and broadcast timers so they run once globally.

SECTION 3 — TRADING ENGINE & OMS

The heart of the project. Expect deep questions on correctness and money-safety.

Q7. Walk me through what happens when a user places a buy order.

BrokerGateway.placeOrder runs pre-flight validation (symbol present, quantity > 0, action is BUY/SELL). For paper trades it delegates to PaperTradingEngine.executeOrder, which: checks the kill-switch and market session; loads the user's account/positions into memory; fetches the authoritative price from MarketDataService; applies a slippage model; runs risk guards (sufficient funds, max-position-size 25% of capital, no short-selling); mutates the in-memory balance and recomputes the weighted average price; then queues the order, account, and position for an async DB flush. It returns a FILLED confirmation immediately.

Follow-up to expect: Where can this race, and where can it lose data?

Q8. How do you compute the new average price on a buy?

New average = (existing quantity x existing average + new fill value) / total quantity. This is the cost-basis weighted average, so selling later realizes P&L against the correct basis. On a sell I keep the average unchanged and add (fill price - average) x quantity to realizedPnl.

Q9. How do you keep wallet accounting correct?

The design intent is a ledger model: balances should be derived from atomic increments plus a transaction log rather than blind overwrites, so the balance is auditable and reconstructable. In the paper engine the account is the source of truth and trades adjust it; for the real Razorpay wallet, funds are only credited after verifying the HMAC-SHA256 payment signature server-side, so a client cannot spoof an amount.

Follow-up to expect: How would you guarantee a deposit is never double-credited?

SECTION 4 — SECURITY & AUTHENTICATION

Be honest here — the project has real gaps, and naming them is a strength.

Q10. How does authentication work in your app?

There are two paths. The Web3 path is solid: the client requests a nonce, signs it with their wallet, and the server recovers the address with `ethers.verifyMessage` and compares it — on success it issues a 7-day JWT and rotates the nonce to prevent replay. The Web2 path is currently a mock: `/login` does not verify a password yet. My next step is to make Web2 real with bcrypt-hashed passwords, or commit fully to the wallet flow.

Follow-up to expect: Why is signature-based auth replay-resistant, and where could it still fail?

Q11. What protects your API at the edge?

helmet for security headers, express-mongo-sanitize against NoSQL injection, and tiered express-rate-limit: a global 500/15min, a strict 20/15min on auth, 50/15min on wallet, and 60/min on trades. CORS is allow-listed. Body size is capped at 10MB. There is a global error handler that hides stack traces in production.

Q12. How does Razorpay signature verification prevent fraud?

After checkout, the client sends `order_id`, `payment_id`, and signature. The server recomputes HMAC-SHA256 over ``order_id|payment_id`` with the Razorpay secret and compares it (ideally with a constant-time compare) to the received signature. Because the secret never leaves the server, a client cannot forge a valid signature for an amount it did not pay.

SECTION 5 — DATABASE & STATE

Schema design, indexing, and the in-memory-vs-durable trade-off.

Q13. How is your data modelled and indexed?

Core collections are User, PaperAccount, PaperPosition, and PaperOrder. Positions carry a compound unique index on { userId, symbol } so each user has one row per symbol and upserts are atomic; orders are indexed on userId for fast history lookups. Mongoose schemas give flexible, evolving documents which suits a fast-moving feature set.

Q14. You execute in memory and flush every 5s. Why, and what is the risk?

In-memory mutation makes fills effectively instant, which is the point of a trading simulator. The risk is durability: a crash between flushes loses up to 5s of trades — the code even comments on this. The mitigation I would add is write-ahead logging the order before mutating, using atomic \$inc on the account as the durable source of truth, and re-queueing failed flushes instead of dropping them.

SECTION 6 — AI & BLOCKCHAIN

The differentiators — know exactly how they work and their limits.

Q15. How does the AI copilot work and how do you keep it reliable?

The /copilot/chat route prompts Gemini 2.5 Flash to return a strict JSON verdict (BUY/SELL/HOLD, confidence, target, rationale). I extract the JSON with a regex and fall back to a mock response if no API key is set. Today it is not grounded in the user's real portfolio, and JSON parsing is fragile — my improvement is schema validation (reject/retry on malformed output), grounding the prompt in the user's live holdings and prices, and a per-user rate limit plus cost cap.

Follow-up to expect: What stops the model from hallucinating a price?

Q16. What does the smart contract add that a database cannot?

PredictionLogger stores a ticker, a prediction hash, a timestamp, and the author on Polygon, and rejects duplicate hashes. The value is tamper-proof provenance: a user can prove they made a specific prediction at a specific time without trusting our database, because nobody — including us — can backdate or alter an on-chain record. That enables a verifiable public track record, which a centralized DB cannot offer credibly.

SECTION 7 — SCALING & PRODUCTION

Senior-leaning questions. Always end on the 100x-scale answer.

Q17. How would you scale this to 50,000 concurrent users?

Three moves. (1) Stateless API behind a load balancer with a Redis Socket.IO adapter so broadcasts fan out across instances. (2) Move shared state (price cache, paper accounts) out of process singletons into Redis, and run the polling/broadcast timers in one dedicated worker so they fire once globally. (3) Replace Yahoo polling with a licensed streaming feed, and shard MongoDB / add read replicas for portfolio reads. Add a queue (e.g. BullMQ) for order processing to smooth bursts.

Q18. How do you know the system is healthy in production?

/api/health/deep reports DB connection, market-data freshness, WebSocket client count, and memory. /api/health/market shows per-symbol price age and staleness. MarketDataService tracks provider success rate and backoff state. The gaps I would close are error tracking (Sentry), real metrics/alerting (Prometheus + Grafana), and a public status page.

SECTION 8 — HARD QUESTIONS ABOUT YOUR CODE

The make-or-break section. These are the flaws a sharp reviewer WILL find. Owning them — with the fix — is the single best thing you can do in the interview.

Q19. I looked at `requirePremium` — it sets `user.isPremium = true` for non-premium users. Isn't your payroll broken?

Yes, and you are right to flag it. That line was a development shortcut so I could test premium pages without a paid account, and it leaked into the main path. It means every authenticated user gets premium. The fix is to delete the auto-upgrade and return 403 when `!user.isPremium`, and to strictly guard the dev/test bypass behind `NODE_ENV` so it can never run in production. I treat this as a P0 bug.

Q20. Your order idempotency key includes `Date.now()`. Doesn't that mean it never deduplicates?

Correct — that is a real bug. Because the timestamp makes every key unique, the duplicate-detection Set never matches, so a double-submit places two orders. The right design is a client-supplied idempotency key (a UUID generated once per user action) sent in a header; the server keys the cache on that and rejects repeats within a window. I caught this reviewing my own code and it is on my fix list.

Q21. Your microstructure order book and "Block Deal" alerts — are those real market data?

No. The order book, the trade tape, and the smart alerts are generated from counters and `Math.random` in `index.js`. They exist to demonstrate the UI and the socket plumbing. I should label them clearly as "Simulated" — unlabelled synthetic signals in a finance product are a trust and compliance problem. The real feeds (price quotes) come from Yahoo Finance and are separate.

Q22. You hard-coded a JWT secret in `docker-compose.yml`. What is the impact?

Anyone with repo access gets the signing key and can forge valid tokens for any user — a critical issue. It also means the JWT fallback string in code could silently be used if the env var is missing. The fix: load secrets from a secret manager, fail fast (exit) if `JWT_SECRET` is unset rather than falling back, rotate the leaked secret, and scrub it from git history.

Q23. There are no automated tests. How do you know the ledger math is correct?

Honestly, right now I rely on manual testing and ad-hoc scripts, which is insufficient for money-like state. The highest-value tests are unit tests on the buy/sell average-price and realized-P&L math, and on the risk guards, plus one e2e happy path — all wired into a CI gate. That is my next deliverable, because the OMS is exactly where a silent bug is most expensive.

Q24. Two orders arrive for the same user at once. Can they corrupt the balance?

There is a window. `executeOrder` awaits `loadUserContext`, and Node yields at that await, so two interleaved orders could both read a stale in-memory balance before either writes back. Within a single tick it is safe, but across awaits it is not truly serialized. The fix is a per-user mutex/queue so a user's orders execute one at a time, and treating an atomic DB \$inc as the authoritative balance check.

SECTION 9 — BEHAVIORAL / PROJECT STORY

They want the human story: judgment, learning, and ownership.

Q25. What was the hardest bug you fixed in this project?

Prices disagreeing across the app — the chart, the order fill, and the portfolio all showed different numbers because each route produced prices independently. It was hard because it was a design flaw, not a single-line bug. I fixed it by introducing `MarketDataService` as a singleton single-source-of-truth and routing every consumer through it. The lesson: in a system where one value must be consistent everywhere, centralize it early.

Q26. If you restarted this project, what would you do differently?

I would build the trust layer first — real auth, a real paywall, secrets management, and tests — before breadth, instead of building 40 feature pages on a mock foundation. I would also pick one differentiator up front (institutional-grade risk, or the on-chain verifiable track record) and go deep, rather than wide. Breadth impressed people in a demo, but depth and trustworthiness are what make it a product.

Q27. What part of this are you most proud of, technically?

The paper-trading engine. It is not a CRUD wrapper — it thinks like an exchange: in-memory fills for latency, a slippage model, position-size and funds risk guards, a kill-switch, risk-event emission, and an async persistence pipeline. Designing the in-memory-vs-durable trade-off, and knowing exactly where its weaknesses are, taught me more about real systems than any other part.

Final advice

The strongest signal you can send is not "my project is flawless" — it is "I can audit my own systems, I know exactly where they break, and I know how to fix them." This handbook gives you the language for both the strengths and the weaknesses. Memorize the file references; opening the exact line when asked is far more convincing than describing it from memory. Good luck.